

Resolução Detalhada e Fundamentada

— Exame de Paradigmas de Programação

Época Normal | Ano Letivo: 2025/2026

Instituição: P.PORTO — Escola Superior de Tecnologia e Gestão

Unidade Curricular: Paradigmas de Programação (PP)

Curso: LEI / LSIRC

Introdução

Este documento apresenta uma resolução exaustiva e detalhada de cada uma das perguntas do exame de Paradigmas de Programação (Época Normal 2025/2026). Para cada questão, além da resposta e do código completo, é incluída uma fundamentação teórica pormenorizada que explica **por que** determinada solução foi adotada, quais as alternativas e os erros comuns a evitar.

PARTE 1 – Perguntas Teóricas

Pergunta 1 (1,5 valores)

Explique detalhadamente as diferenças entre classes abstratas e interfaces em Java. Em que situações é mais adequado optar por uma classe abstrata e em que situações é preferível uma interface? Justifique a sua resposta e ilustre cada caso com um exemplo prático que evidencie as características distintivas de cada mecanismo.

Resposta e Comparação Estruturada

A tabela seguinte resume as principais diferenças estruturais entre classes abstratas e interfaces no Java:

Característica	Classe Abstrata	Interface
Herança	Suporta apenas herança simples (uma classe pode estender no máximo uma classe abstrata).	Suporta herança múltipla de tipo (uma classe pode implementar várias interfaces).
Atributos de Estado	Pode ter variáveis de instância (campos com qualquer visibilidade: <code>private</code> , <code>protected</code> , <code>public</code>).	Não possui variáveis de instância. Todos os atributos são implicitamente <code>public static final</code> (constantes).
Construtores	Pode ter construtores , que são invocados pelas subclasses (<code>super(...)</code>) para inicializar o estado herdado.	Não possui construtores , pois não pode reter estado mutável nem ser instanciada diretamente.
Métodos concretos	Pode definir métodos com corpo (concretos) e métodos abstratos (<code>abstract</code>).	Antes do Java 8, apenas métodos abstratos. Desde o Java 8, permite métodos <code>default</code> e <code>static</code> . Desde o Java 9, métodos <code>private</code> .
Intenção de Design	Define uma relação de identidade ("É UM" / <i>is-a</i>).	Define um contrato de comportamento ou capacidade ("CONSEGUE FAZER" / <i>can-do</i>).

Quando escolher cada mecanismo?

1. Optar por Classe Abstrata quando:

- Existe a necessidade de **partilhar estado (campos não-constantes)** comuns a várias classes relacionadas.
- Queremos fornecer uma implementação base de métodos que as subclasses possam herdar ou redefinir, evitando a duplicação de código.
- As classes derivadas têm uma relação clara de identidade ("É UM") com a superclasse (ex: um `Cão` é um `Animal`).
- Queremos controlar o acesso de visibilidade dos membros comuns utilizando modificadores como `protected` ou `private`.

6. Optar por Interface quando:

- Pretendemos definir um **contrato de comportamento** que pode ser partilhado por classes completamente diferentes e sem qualquer relação na árvore de herança (ex: um `Documento` e uma `Imagem` podem ser ambos `Imprimivel`).
- Precisamos de usufruir de **herança múltipla** para que uma classe herde múltiplas especificações de comportamento.

9. Queremos desenhar uma arquitetura desacoplada baseada em contratos ("programar para interfaces, não para implementações").
-

Exemplos Práticos Ilustrativos

Exemplo de Classe Abstrata: Hierarquia de Veículos

Aqui, a classe abstrata é adequada porque todos os veículos partilham campos de estado mutáveis (`matricula`, `combustivel`) e um construtor comum para os inicializar.

```
// Classe Abstrata: Representa a base "É UM"
public abstract class Veiculo {
    private String matricula;
    private double nivelCombustivel; // Estado partilhado pelas subclasses

    // Construtor: Classes abstratas podem (e devem) ter construtores para o seu estado
    public Veiculo(String matricula) {
        this.matricula = matricula;
        this.nivelCombustivel = 100.0; // Inicia atestado
    }

    public String getMatricula() {
        return matricula;
    }

    // Método concreto comum
    public void abastecer(double quantidade) {
        this.nivelCombustivel += quantidade;
    }

    // Método abstrato: Cada subclasse calcula a autonomia à sua maneira
    public abstract double calcularAutonomia();
}

// Subclasse concreta
public class Carro extends Veiculo {
    private double consumoMedio;

    public Carro(String matricula, double consumoMedio) {
        super(matricula); // Invocação do construtor da classe abstrata
        this.consumoMedio = consumoMedio;
    }

    @Override
    public double calcularAutonomia() {
        // Implementação específica para Carro
        return (100.0 / consumoMedio) * 50; // Exemplo hipotético
    }
}
```

Exemplo de Interface: Contrato de Exportação

Aqui, a interface é adequada porque qualquer classe de qualquer hierarquia (ex: `Relatorio` ou `Fatura`) pode necessitar de ser exportada para PDF, sem que tenham de partilhar a mesma classe pai.

```
// Interface: Define o comportamento "CONSEGUE FAZER"
public interface ExportavelPDF {
    // Método abstrato implícito (contrato)
    byte[] exportarParaPDF();

    // Método default (Java 8+): comportamento padrão sem obrigar a implementar
    default void logExportacao() {
        System.out.println("Exportando documento para formato PDF padrão...");
    }
}

// Classe 1: Pertence à hierarquia de Documentos de Texto
public class Relatorio implements ExportavelPDF {
    private String conteudo;

    @Override
    public byte[] exportarParaPDF() {
        // Lógica de conversão do texto do relatório para PDF
        return conteudo.getBytes();
    }
}

// Classe 2: Pertence à hierarquia de Transações Financeiras
public class Fatura implements ExportavelPDF {
    private double valor;

    @Override
    public byte[] exportarParaPDF() {
        // Lógica de conversão dos dados da fatura para PDF
        return ("Fatura no valor de " + valor).getBytes();
    }
}
```

Raciocínio de Escolha e Alternativas:

- **Por que não usar uma classe concreta em vez da abstrata?** Se usássemos uma classe concreta (ex: `Veiculo`), poderíamos criar instâncias diretas dela (`new Veiculo()`). Isso violaria o modelo de negócio, pois um "veículo genérico" não existe na realidade — existem carros, motos, caminhões. A classe abstrata impede esta instanciação incorreta em tempo de compilação.
- **Por que não usar apenas interfaces?** Se usássemos a interface `Veiculo`, não poderíamos declarar as variáveis de instância `matricula` e `nivelCombustivel`. Teríamos de redefinir e reimplementar estas variáveis e os seus respetivos métodos de acesso (`abastecer`, `getMatricula`) em **todas** as subclasses, duplicando código desnecessariamente.

Pergunta 2 (1,5 valores)

Descreva detalhadamente o modo como Java realiza a passagem de argumentos para os métodos, distinguindo o comportamento aplicado a tipos primitivos do comportamento aplicado a referências de objetos. Esclareça os equívocos frequentes associados a este mecanismo e fundamente a sua explicação com exemplos concretos que demonstrem os efeitos sobre os valores e os estados dos objetos.

Resposta e Funcionamento da JVM

Em Java, **todas as passagens de argumentos são efetuadas estritamente por valor (pass-by-value)**. Não existe passagem por referência na linguagem Java. O "valor" que é copiado e passado depende do tipo da variável:

1. **Tipos Primitivos** (`int`, `double`, `boolean`, etc.):
 2. O valor armazenado na variável é o próprio dado numérico/lógico.
 3. Quando o argumento é passado para o método, a JVM cria uma **cópia exata do dado** e coloca-a na variável local do método (no frame de execução da pilha - *Stack*).
 4. Qualquer alteração efetuada a esta variável local dentro do método ocorre apenas sobre a cópia e **não tem qualquer impacto** na variável original.
5. **Tipos de Referência** (Objetos, Arrays, etc.):
 6. O valor armazenado na variável não é o objeto em si, mas sim o **endereço de memória (referência)** que aponta para o local da memória *Heap* onde o objeto realmente reside.

7. Quando um objeto é passado como argumento, a JVM cria uma **cópia do endereço de memória (cópia da referência)** e atribui-a ao parâmetro do método.
 8. **Consequência 1 (Alteração de Estado):** Como o parâmetro local e a variável original contêm cópias do mesmo endereço de memória, ambos apontam para o *mesmo objeto físico na Heap*. Deste modo, qualquer alteração ao estado interno do objeto (ex: alteração de atributos ou adição de elementos) efetuada dentro do método será visível fora dele.
 9. **Consequência 2 (Reatribuição de Referência):** Se o método reatribuir o parâmetro (ex: `parametro = new Objeto()`), a cópia local passará a apontar para outro endereço de memória. A referência original fora do método continua a apontar inalterada para o objeto inicial. Logo, reatribuir o objeto dentro do método não afeta a variável original.
-

Exemplo Prático e Explicação Linha a Linha

O código seguinte ilustra os diferentes comportamentos de passagem de parâmetros:

```
public class TestePassagem {

    // Método que tenta alterar um tipo primitivo
    public static void alterarPrimitivo(int numero) {
        numero = 999; // Modifica apenas a cópia local armazenada na stack deste método
    }

    // Método que modifica o estado interno de um objeto (através da cópia da referência)
    public static void modificarEstadoObjeto(int[] array) {
        array[0] = 50; // Modifica o objeto real que está na Heap
    }

    // Método que tenta reatribuir a referência a um novo objeto
    public static void reatribuirReferenciaObjeto(int[] array) {
        array = new int[]{100, 200, 300}; // Reatribui apenas a referência local (cópia)
        array[0] = 999; // Altera o novo array criado na Heap, não o original
    }

    public static void main(String[] args) {
        // 1. Teste de Primitivos
        int a = 10;
        alterarPrimitivo(a);
        System.out.println("Primitivo 'a': " + a); // Imprime 10. A variável 'a' ficou intacta.

        // 2. Teste de Estado de Objeto
        int[] meuArray = {1, 2, 3};
        modificarEstadoObjeto(meuArray);
        System.out.println("meuArray[0]: " + meuArray[0]); // Imprime 50. O estado do objeto foi modi

        // 3. Teste de Reatribuição de Referência
        reatribuirReferenciaObjeto(meuArray);
        System.out.println("meuArray[0] após reatribuição: " + meuArray[0]); // Imprime 50, e não 999
    }
}
```

Esclarecimento do Equívoco Frequente

O equívoco mais comum em Java é a afirmação: *"Os tipos primitivos são passados por valor e os objetos são passados por referência."*

Esta afirmação está **incorreta**. Se os objetos fossem passados por referência, o método

`reatribuirReferenciaObjeto` teria alterado a referência `meuArray` no método `main`, fazendo com que ela apontasse para o novo array `{100, 200, 300}`, e o output final seria `999`. Como o output permaneceu `50`, fica provado que apenas a **cópia da referência** foi passada e manipulada localmente.

Pergunta 3 (1,5 valores)

Explique o conceito de conversão de tipos (casting) no contexto da herança e do polimorfismo. Discuta os riscos associados a conversões incorretas e as situações em que cada tipo de conversão é apropriado. Ilustre com um exemplo prático que demonstre as conversões.

Resposta e Conceitos de Casting

Em Java, a compatibilidade de tipos é verificada de forma estrita em tempo de compilação. Contudo, devido ao polimorfismo, uma variável de uma superclasse pode apontar para um objeto de uma subclasse. O **casting** é o mecanismo utilizado para instruir explicitamente o compilador a tratar um objeto como sendo de outro tipo na hierarquia. Existem dois tipos de conversão:

1. Upcasting (Conversão Ascendente)

- **Definição:** Conversão de uma referência de uma subclasse para uma superclasse (subir na árvore de herança).
- **Segurança:** É **sempre seguro** e automático (conversão implícita), dispensando operadores especiais.
- **Justificação:** Pelo Princípio da Substituição de Liskov, uma subclasse é uma especialização da superclasse, logo, contém todos os membros desta.
- **Consequência:** Perde-se temporariamente o acesso direto aos métodos específicos da subclasse. O compilador apenas permite chamar os métodos declarados na superclasse, embora a JVM execute a versão sobreposta da subclasse (polimorfismo).

2. Downcasting (Conversão Descendente)

- **Definição:** Conversão de uma referência de uma superclasse para uma subclasse (descer na árvore de herança).
- **Segurança:** É **potencialmente perigoso** e requer conversão explícita: `(SubClasse) referencia`.

- **Risco:** O compilador permite o downcasting desde que haja uma relação de herança entre as classes. No entanto, se o objeto real que está na memória Heap em tempo de execução não for compatível com o tipo pretendido (ou seja, se tentarmos forçar um objeto de um tipo A a comportar-se como tipo B sem que o seja), a JVM lançará a exceção `ClassCastException`, interrompendo a execução.
- **Mitigação:** Deve-se utilizar o operador `instanceof` para verificar a compatibilidade antes de efetuar o downcasting.

Exemplo Prático Comentado

Considere as classes `Funcionario` e `Programador` (onde `Programador extends Funcionario`):

```
public class Funcionario {  
    private String nome;  
  
    public Funcionario(String nome) {  
        this.nome = nome;  
    }  
  
    public void trabalhar() {  
        System.out.println(nome + " está a trabalhar genericamente.");  
    }  
}  
  
public class Programador extends Funcionario {  
    public Programador(String nome) {  
        super(nome);  
    }  
  
    @Override  
    public void trabalhar() {  
        System.out.println("Escrevendo código em Java.");  
    }  
  
    public void programar() {  
        System.out.println("Efetuando commit no repositório.");  
    }  
}
```

No programa principal, analisamos as conversões:

```
public class TesteCasting {  
    public static void main(String[] args) {  
        // --- 1. UPCASTING ---  
        // Programador é um Funcionario. O compilador faz a conversão implicitamente.  
        Funcionario func = new Programador("Alice");  
  
        func.trabalhar(); // Polimorfismo: Executa a versão de Programador ("Escrevendo código em Java")  
        // func.programar(); // ERRO DE COMPILAÇÃO: O tipo de referência Funcionario não conhece o método programar()  
  
        // --- 2. DOWNCASTING SEGURO ---  
        // Sabemos que 'func' aponta para um objeto Programador na Heap.  
        // Usamos o 'instanceof' para validar antes do cast.  
        if (func instanceof Programador) {  
            Programador prog = (Programador) func; // Cast explícito  
            prog.programar(); // OK: Agora temos acesso ao método específico da subclasse  
        }  
  
        // --- 3. DOWNCASTING INCORRETO (Risco) ---  
        Funcionario funcReal = new Funcionario("Carlos"); // Objeto real é apenas Funcionario, não Programador  
  
        try {  
            // O compilador aceita porque existe relação de herança, mas em runtime falhará!  
            Programador progIncorreto = (Programador) funcReal;  
            progIncorreto.programar();  
        } catch (ClassCastException e) {  
            System.out.println("Erro capturado: Não é possível converter um Funcionario base num Programador  
            // Lança ClassCastException porque o objeto real 'Carlos' não possui o estado/comportamento de Programador  
        }  
    }  
}
```

Pergunta 4 (1,5 valores)

Distinga os conceitos de identidade e de igualdade de objetos em Java, esclarecendo a diferença entre o operador `==` e o método `equals()`. Explique igualmente o papel do método `toString()`. Forneça um exemplo prático que demonstre a redefinição correta dos métodos `equals()` e `toString()` numa classe.

Resposta: Identidade vs Igualdade

- **Identidade de Objetos (`==`):**
 - Refere-se à **posição física de memória** do objeto.
 - O operador `==` compara os endereços de memória das referências. Avalia se duas variáveis apontam exatamente para a mesma instância física na Heap (se `ref1` e `ref2` são o mesmo objeto).
 - Para tipos primitivos, compara diretamente o valor binário do dado.
- **Igualdade de Objetos (`equals()`):**
 - Refere-se à **equivalência semântica ou lógica** de dados.
 - O método `equals(Object obj)` é herdado da classe base `Object`. Por defeito, a sua implementação nativa faz uma comparação de identidade (`this == obj`).
 - Para que as nossas classes possam definir o que constitui "conteúdo idêntico" (ex: dois estudantes são logicamente o mesmo se tiverem o mesmo número de identificação civil), é necessário **redirecionar/sobrepôr** (`@Override`) este método.

O papel do método `toString()`

O método `toString()` é herdado de `Object` e serve para obter uma **representação legível em formato de texto (String)** do estado de um objeto.

A implementação por defeito retorna `NomeDaClasse@hashcodeHexadecimal`, o que não é útil. * Redefinir este método é uma excelente prática para facilitar a depuração (debugging*) e a geração de relatórios de log, sendo executado automaticamente quando passamos o objeto a `System.out.println()` ou concatenamos com texto.

Exemplo Prático de Redefinição Correta (`equals` e `toString`)

Abaixo é apresentada a classe `Estudante`. A igualdade lógica é baseada exclusivamente no atributo `numeroEstudante`.

```
import java.util.Objects;

public class Estudante {
    private String numeroEstudante;
    private String nome;

    public Estudante(String numeroEstudante, String nome) {
        this.numeroEstudante = numeroEstudante;
        this.nome = nome;
    }

    // REDEFINIÇÃO DO EQUALS
    @Override
    public boolean equals(Object obj) {
        // Passo 1: Otimização por Identidade
        // Se apontam para o mesmo local de memória, são obrigatoriamente iguais.
        if (this == obj) {
            return true;
        }

        // Passo 2: Verificação de Nulo
        // Se o objeto comparado for nulo, a resposta é sempre false.
        if (obj == null) {
            return false;
        }

        // Passo 3: Verificação de Tipo Exato
        // Garantimos que os objetos pertencem exatamente à mesma classe.
        // Nota: O uso de getClass() é preferível ao 'instanceof' neste caso para
        // garantir a simetria se existirem subclasses (evita problemas polimórficos).
        if (this.getClass() != obj.getClass()) {
            return false;
        }

        // Passo 4: Downcasting Seguro
        Estudante outro = (Estudante) obj;

        // Passo 5: Comparação lógica dos atributos chave
        // Usamos Objects.equals() para evitar NullPointerException se algum campo for null.
        return Objects.equals(this.numeroEstudante, outro.numeroEstudante);
    }
}
```

```
}

// REDEFINIÇÃO DO HASHCODE (Obrigatório ao redefinir o equals)
@Override
public int hashCode() {
    // Se dois objetos são iguais pelo equals(), devem retornar o mesmo hashCode.
    return Objects.hash(numeroEstudante);
}

// REDEFINIÇÃO DO TOSTRING
@Override
public String toString() {
    return "Estudante [Nº: " + numeroEstudante + " | Nome: " + nome + " ]";
}
}
```

PARTE 2 – Programação em Java

Pergunta 1a (3 valores)

Considere a interface `RefrigeratedVehicle` que representa um veículo refrigerado com um limite máximo de quilómetros. Implemente a interface numa classe denominada `RefrigeratedVehicleImpl`. O veículo deve possuir um estado (`ENABLED`, `DISABLED`) e deve ser inicializado como `ENABLED` por defeito. Para a implementação do método `equals`, considere que duas instâncias de `RefrigeratedVehicle` são iguais se possuírem o mesmo código (devolvido através do método `getCode()`).

Definição do Enum de Estado

Para garantir a tipagem forte do estado, cria-se o enum `VehicleState`:

```
public enum VehicleState {
    ENABLED,
    DISABLED
}
```

Implementação da Classe RefrigeratedVehicleImpl

```
import java.util.Objects;

/**
 * Classe que implementa o contrato de um veículo refrigerado.
 * Como RefrigeratedVehicle estende Vehicle, esta classe deve implementar
 * todas as operações definidas em ambas as interfaces.
 */
public class RefrigeratedVehicleImpl implements RefrigeratedVehicle {
    private final String code;
    private final ItemType supplyType;
    private final double maxCapacity;
    private final double maxKilometers;
    private VehicleState state; // Variável de estado mutável (não final)

    /**
     * Construtor completo para inicializar o veículo.
     * O estado é configurado como ENABLED por defeito, conforme exigido.
     */
    public RefrigeratedVehicleImpl(String code, ItemType supplyType, double maxCapacity, double maxKi
        if (code == null) {
            throw new IllegalArgumentException("O código do veículo não pode ser nulo.");
        }
        this.code = code;
        this.supplyType = supplyType;
        this.maxCapacity = maxCapacity;
        this.maxKilometers = maxKilometers;
        this.state = VehicleState.ENABLED; // Inicializado como ENABLED por defeito
    }

    // --- MÉTODOS DA INTERFACE VEHICLE ---
    @Override
    public String getCode() {
        return this.code;
    }

    @Override
    public ItemType getSupplyType() {
        return this.supplyType;
    }
}
```



```
@Override
public double getMaxCapacity() {
    return this.maxCapacity;
}

// --- MÉTODOS DA INTERFACE REFRIGERATEDVEHICLE ---
@Override
public double getMaxKilometers() {
    return this.maxKilometers;
}

// --- GESTÃO DE ESTADO (GETTER E SETTER) ---
public VehicleState getState() {
    return this.state;
}

public void setState(VehicleState state) {
    if (state == null) {
        throw new IllegalArgumentException("O estado do veículo não pode ser nulo.");
    }
    this.state = state;
}

// --- SOBREPOSIÇÃO DO EQUALS (Contrato de Negócio) ---
@Override
public boolean equals(Object obj) {
    // Comparação de identidade física
    if (this == obj) {
        return true;
    }
    // Validação de nulidade
    if (obj == null) {
        return false;
    }
    // Comparação de tipo amplo:
    // O enunciado estipula que DUAS instâncias de RefrigeratedVehicle são iguais
    // se tiverem o mesmo código. Usamos 'instanceof' para permitir comparação
    // entre diferentes implementações ou subclasses da interface RefrigeratedVehicle.
    if (!(obj instanceof RefrigeratedVehicle)) {
        return false;
    }
}
```

```
    }

    RefrigeratedVehicle other = (RefrigeratedVehicle) obj;
    // String.equals() para comparar os códigos
    return this.code.equals(other.getCode());
}

// --- SOBREPOSIÇÃO DO HASHCODE ---
@Override
public int hashCode() {
    // Garantimos que a geração do hash do objeto se baseia apenas no código,
    // em perfeita simetria com a lógica do método equals.
    return Objects.hash(this.code);
}

// --- SOBREPOSIÇÃO DO TOSTRING ---
@Override
public String toString() {
    return "Veículo Refrigerado [" +
        "Código: " + code +
        " | Tipo: " + supplyType +
        " | Capacidade Máx: " + maxCapacity + "kg" +
        " | Distância Máx: " + maxKilometers + "km" +
        " | Estado: " + state +
        "];"
}
}
```

Fundamentação Pormenorizada da Implementação:

1. Por que usar `instanceof RefrigeratedVehicle` em vez de `getClass()` no `equals` ?

O enunciado diz expressamente: *"considere que duas instâncias de `RefrigeratedVehicle` são iguais..."*. Se usássemos `this.getClass() != obj.getClass()`, estaríamos a restringir a igualdade apenas a instâncias da classe concreta `RefrigeratedVehicleImpl`. Se houvesse outra classe (ex: `AdvancedRefrigeratedVehicleImpl`) que implementasse a mesma interface, o `equals` retornaria `false` mesmo com códigos idênticos. O `instanceof` garante a conformidade com a especificação da interface.

2. Por que foi adicionado o `hashCode()` se o enunciado pedia apenas o `equals` ?

Sempre que se redefine o `equals` em Java, é um requisito estrito da linguagem redefinir o `hashCode()`. Se não o fizéssemos, o código continuaria a compilar, mas qualquer coleção que

use tabelas de dispersão (como `HashSet<Vehicle>` ou `HashMap<Vehicle, Route>`) falharia ao detetar duplicados, violando o comportamento lógico do sistema.

3. Uso de `final` nos atributos:

Atributos como `code`, `supplyType`, `maxCapacity` e `maxKilometers` foram declarados como `final` para evidenciar a sua **imutabilidade**. Uma vez criado o veículo, estes dados não se alteram. O único atributo não-final é o `state`, pois o estado do veículo pode transitar de ativo para inativo.

Pergunta 1b (2 valores)

Desenvolva o código necessário para testar a classe implementada (por exemplo, no contexto de um método `main`). Apresente um exemplo de teste para cada método implementado.

Classe de Teste

```
public class TestRefrigeratedVehicle {  
    public static void main(String[] args) {  
        System.out.println("===== INÍCIO DOS TESTES UNITÁRIOS =====\n");  
  
        // 1. Instanciação e Construtor  
        // Criamos instâncias com dados idênticos e diferentes para validar o equals  
        RefrigeratedVehicleImpl v1 = new RefrigeratedVehicleImpl("V-001", ItemType.PERISHABLE_FOOD, 150.0, 800.0, 3);  
        RefrigeratedVehicleImpl v2 = new RefrigeratedVehicleImpl("V-001", ItemType.MEDICINE, 800.0, 3, 150.0);  
        RefrigeratedVehicleImpl v3 = new RefrigeratedVehicleImpl("V-002", ItemType.PERISHABLE_FOOD, 150.0, 800.0, 3);  
  
        // 2. Testar Getters da classe Vehicle (implementados por herança)  
        System.out.println("--- Teste de Getters (Vehicle) ---");  
        System.out.println("v1 - Código esperado 'V-001': " + v1.getCode());  
        System.out.println("v1 - Tipo de Carga esperado 'PERISHABLE_FOOD': " + v1.getSupplyType());  
        System.out.println("v1 - Capacidade Máx esperada '1000.0': " + v1.getMaxCapacity());  
        System.out.println();  
  
        // 3. Testar Getters específicos de RefrigeratedVehicle  
        System.out.println("--- Teste de Getters (RefrigeratedVehicle) ---");  
        System.out.println("v1 - Distância Máx esperada '150.0': " + v1.getMaxKilometers());  
        System.out.println();  
  
        // 4. Testar Estado Inicial e Modificação de Estado  
        System.out.println("--- Teste de Estado do Veículo ---");  
        System.out.println("Estado inicial esperado (ENABLED): " + v1.getState());  
        v1.setState(VehicleState.DISABLED);  
        System.out.println("Estado após alteração (DISABLED): " + v1.getState());  
        v1.setState(VehicleState.ENABLED); // Repor para os testes seguintes  
        System.out.println();  
  
        // 5. Testar Métodos equals() e hashCode()  
        System.out.println("--- Teste do Método equals() ---");  
        // Caso A: Mesma Referência física  
        System.out.println("A. v1.equals(v1) [Esperado: true]: " + v1.equals(v1));  
  
        // Caso B: Conteúdo lógico idêntico (mesmo código, outros atributos diferentes)  
        System.out.println("B. v1.equals(v2) [Esperado: true]: " + v1.equals(v2));  
  
        // Caso C: Conteúdo lógico diferente (códigos diferentes)
```

```
System.out.println("C. v1.equals(v3) [Esperado: false]: " + v1.equals(v3));

// Caso D: Comparação com nulo
System.out.println("D. v1.equals(null) [Esperado: false]: " + v1.equals(null));

// Caso E: Comparação com tipo incompatível
System.out.println("E. v1.equals(\"String Qualquer\") [Esperado: false]: " + v1.equals("String Qualquer"));

System.out.println("\n--- Teste do Método hashCode() ---");
System.out.println("Hash de v1: " + v1.hashCode());
System.out.println("Hash de v2 (deve ser idêntico ao v1): " + v2.hashCode());
System.out.println("Hash de v3 (deve ser diferente): " + v3.hashCode());
System.out.println("v1.hashCode() == v2.hashCode() [Esperado: true]: " + (v1.hashCode() == v2.hashCode()));
System.out.println();

// 6. Testar Método toString()
System.out.println("--- Teste do Método toString() ---");
System.out.println("Representação textual de v1:");
System.out.println(v1.toString());

System.out.println("\n===== FIM DOS TESTES =====");
}
}
```

Pergunta 2a (4 valores)

Implemente os seguintes métodos na classe `StrategyImpl` que podem ser utilizados para a geração da rota: *

```
boolean hasCollectableContainer(Vehicle vehicle, AidBox aidbox); *
```

```
boolean addAidBoxToRoute(Route route, AidBox aidbox, RouteValidator validator);
```

Implementação de `hasCollectableContainer` **e** `addAidBoxToRoute`

```
public class StrategyImpl implements Strategy {

    // --- Outros atributos e construtores se necessário ---

    /**
     * Verifica se a AidBox tem pelo menos um container do mesmo tipo que o veículo
     * cuja última medição é superior a 80% da sua capacidade.
     *
     * @param vehicle Veículo de recolha
     * @param aidbox Caixa de ajuda a avaliar
     * @return true se contiver pelo menos um contentor apto a recolha, false caso contrário.
     */
    public boolean hasCollectableContainer(Vehicle vehicle, AidBox aidbox) {
        // Validação defensiva de argumentos
        if (vehicle == null || aidbox == null) {
            return false;
        }

        // Obter a lista de contentores da caixa de ajuda
        Container[] containers = aidbox.getContainers();
        if (containers == null) {
            return false;
        }

        // Iteração pelo array de contentores
        for (int i = 0; i < containers.length; i++) {
            Container currentContainer = containers[i];

            // Ignorar contentores nulos no array
            if (currentContainer == null) {
                continue;
            }

            // Regra 1: O tipo do contentor deve ser igual ao tipo de carga do veículo.
            // Como ItemType é um Enum, a comparação direta com '==' é correta e segura.
            if (currentContainer.getType() == vehicle.getSupplyType()) {

                // Obter a última medição
                Measurement lastMeasurement = currentContainer.getLastMeasurement();
```

```
// Validação crítica de nulo para evitar NullPointerException!
if (lastMeasurement != null) {
    double valorMedido = lastMeasurement.getValue();
    double capacidadeTotal = currentContainer.getCapacity();

    // Regra 2: O valor medido deve ser superior a 80% da capacidade total (capacidade
    if (valorMedido > (capacidadeTotal * 0.8)) {
        return true; // Encontrou pelo menos um, cumpre o requisito de curto-circuito
    }
}

}

return false; // Nenhum contentor cumpriu os critérios
}

/**
 * Tenta adicionar uma AidBox à rota após validação.
 *
 * @param route Rota à qual adicionar a AidBox
 * @param aidbox Caixa de ajuda a adicionar
 * @param validator Validador de rota
 * @return true se adicionada com sucesso, false caso contrário.
 */
public boolean addAidBoxToRoute(Route route, AidBox aidbox, RouteValidator validator) {
    // Validação defensiva
    if (route == null || aidbox == null || validator == null) {
        return false;
    }

    // Passo 1: Efetuar validação com o validator
    boolean isValid = validator.validate(route, aidbox);

    if (isValid) {
        try {
            // Passo 2: Adicionar à rota
            // Nota: O método addAidBox declara que lança a exceção RouteException (checked exception)
            route.addAidBox(aidbox);
            return true;
        } catch (RouteException e) {
```

```

        // Se o método addAidBox originar RouteException, capturamos a exceção
        // e retornamos false de forma segura sem crashar a aplicação.
        return false;
    }
}

return false; // Validação falhou
}

@Override
public Route[] generate(IIInstitution inst, RouteValidator validator) {
    // Implementado na alínea 2b
    return new Route[0];
}
}

```

Fundamentação Pormenorizada das Escolhas Algorítmicas:

1. Prevenção contra NullPointerException (NPE) no `hasCollectableContainer` :

Em exames de desenvolvimento de software, um erro extremamente comum é chamar diretamente `containers[i].getLastMeasurement().getValue()`. Se a última medição for nula (ex: sensor avariado ou contendor recém-instalado sem medições), o programa lança NPE e termina. A validação `lastMeasurement != null` é essencial para robustez.

2. Uso do operador `==` para comparar `ItemType` :

Como `ItemType` é um tipo enumerado (`enum`), a JVM garante que existe apenas uma instância na memória para cada valor do enum. Por isso, a comparação com `==` é recomendada, pois além de mais rápida, é nula-segura (não causa NPE se um dos lados for nulo) e garante segurança em tempo de compilação.

3. Tratamento de Exceções em `addAidBoxToRoute` :

A exceção `RouteException` é uma exceção do tipo *checked* (verificada). Isto significa que o compilador obriga a tratá-la. Como a assinatura do método `addAidBoxToRoute` definida no enunciado **não possui** a declaração `throws RouteException`, o programador é obrigado a envolver a chamada num bloco `try-catch`. Capturar a exceção e retornar `false` satisfaz exatamente a especificação de robustez do enunciado.

Pergunta 2b (5 valores)

Na classe `StrategyImpl`, implemente o método `generate`, gerando as rotas necessárias. Regras a considerar: * Para cada veículo devolvido pelo método `getVehicles()` da interface `Institution`, deve ser criada uma nova rota. Assuma que só existe um veículo para cada tipo. * As `AidBoxes` existentes são as devolvidas pelo método `getAidBoxes()` da interface `Institution`. * Deve utilizar os métodos desenvolvidos na alínea anterior. Se não os implementou anteriormente, assuma que os métodos já existem. * O array devolvido pelo método `generate` deve conter rotas com as `AidBoxes` (sem posições nulas ou rotas vazias).

Implementação do Método `generate`

```
@Override
public Route[] generate(IInstitution inst, RouteValidator validator) {
    // Validação defensiva inicial
    if (inst == null || validator == null) {
        return new Route[0]; // Retorna array vazio em vez de null (boa prática API)
    }

    // Obter os veículos e as caixas de ajuda da instituição
    Vehicle[] vehicles = inst.getVehicles();
    AidBox[] aidBoxes = inst.getAidBoxes();

    // Se não houver veículos ou caixas, não é possível gerar rotas
    if (vehicles == null || aidBoxes == null || vehicles.length == 0 || aidBoxes.length == 0) {
        return new Route[0];
    }

    // Como não podemos usar ArrayList, criamos um array temporário com o tamanho máximo possível
    // No pior dos casos (se todos os veículos gerarem rotas válidas), haverá tantos caminhos
    // quantas as instâncias de veículos.
    Route[] tempRoutes = new Route[vehicles.length];
    int routeCount = 0; // Aponta para a próxima posição livre e conta as rotas válidas

    // Iterar sobre cada veículo da instituição
    for (int i = 0; i < vehicles.length; i++) {
        Vehicle currentVehicle = vehicles[i];

        // Ignorar possíveis posições nulas no array de veículos
        if (currentVehicle == null) {
            continue;
        }

        // Regra 1: Criar uma nova rota para cada veículo
        // Pressuposto: existe uma classe concreta RouteImpl que implementa Route
        // e recebe o Vehicle correspondente no construtor para associar a capacidade/tipo.
        Route currentRoute = new RouteImpl(currentVehicle);
        boolean routeHasAidBoxes = false; // Flag para controlar se a rota foi preenchida

        // Iterar sobre todas as caixas de ajuda existentes
        for (int j = 0; j < aidBoxes.length; j++) {
```

```
AidBox currentAidBox = aidBoxes[j];

// Ignorar caixas de ajuda nulas no array
if (currentAidBox == null) {
    continue;
}

// Regra 2: Verificar se a caixa tem contentores recolhíveis para este veículo
if (hasCollectableContainer(currentVehicle, currentAidBox)) {

    // Regra 3: Tentar adicionar a caixa à rota (com validação interna)
    if (addAidBoxToRoute(currentRoute, currentAidBox, validator)) {
        routeHasAidBoxes = true; // Rota passou a conter pelo menos uma caixa
    }
}

// Regra 4: O array devolvido não deve conter rotas vazias
// Só adicionamos a rota ao array temporário se ela contiver pelo menos uma caixa válida
if (routeHasAidBoxes) {
    tempRoutes[routeCount] = currentRoute;
    routeCount++;
}

// Regra 5: O array devolvido não deve conter posições nulas (Shrinking)
// Criamos o array final com o tamanho exato do nosso contador 'routeCount'
Route[] finalRoutes = new Route[routeCount];
for (int i = 0; i < routeCount; i++) {
    finalRoutes[i] = tempRoutes[i];
}

return finalRoutes;
}
```

Classe `StrategyImpl` Completa e Integrada

Abaixo está a classe `StrategyImpl` com todos os métodos integrados e organizados para entrega:

```
public class StrategyImpl implements Strategy {

    /**
     * Auxiliar Pergunta 2a.
     */
    private boolean hasCollectableContainer(Vehicle vehicle, AidBox aidbox) {

        if (vehicle == null || aidbox == null) return false;
        Container[] containers = aidbox.getContainers();
        if (containers == null) return false;

        for (int i = 0; i < containers.length; i++) {
            Container current = containers[i];
            if (current == null) continue;

            if (current.getType() == vehicle.getSupplyType()) {
                Measurement last = current.getLastMeasurement();
                if (last != null) {
                    if (last.getValue() > (current.getCapacity() * 0.8)) {
                        return true;
                    }
                }
            }
        }
        return false;
    }

    /**
     * Auxiliar Pergunta 2a.
     */
    private boolean addAidBoxToRoute(Route route, AidBox aidbox, RouteValidator validator) {

        if (route == null || aidbox == null || validator == null) return false;

        if (validator.validate(route, aidbox)) {
            try {
                route.addAidBox(aidbox);
                return true;
            } catch (RouteException e) {
                return false;
            }
        }
    }
}
```

```
        return false;
    }

    /**
     * Gerador de Rotas Principal (Pergunta 2b).
     */
    @Override
    public Route[] generate(IInstitution inst, RouteValidator validator) {
        if (inst == null || validator == null) {
            return new Route[0];
        }

        Vehicle[] vehicles = inst.getVehicles();
        AidBox[] aidBoxes = inst.getAidBoxes();

        if (vehicles == null || aidBoxes == null || vehicles.length == 0 || aidBoxes.length == 0) {
            return new Route[0];
        }

        Route[] tempRoutes = new Route[vehicles.length];
        int routeCount = 0;

        for (int i = 0; i < vehicles.length; i++) {
            Vehicle currentVehicle = vehicles[i];
            if (currentVehicle == null) {
                continue;
            }

            // Criação da rota com base no pressuposto da existência de RouteImpl
            Route currentRoute = new RouteImpl(currentVehicle);
            boolean routeHasAidBoxes = false;

            for (int j = 0; j < aidBoxes.length; j++) {
                AidBox currentAidBox = aidBoxes[j];
                if (currentAidBox == null) {
                    continue;
                }

                if (hasCollectableContainer(currentVehicle, currentAidBox)) {
                    if (addAidBoxToRoute(currentRoute, currentAidBox, validator)) {
                        routeHasAidBoxes = true;
                    }
                }
            }

            tempRoutes[routeCount] = currentRoute;
            routeCount++;
        }

        return tempRoutes;
    }
}
```

```
        }
    }
}

if (routeHasAidBoxes) {
    tempRoutes[routeCount] = currentRoute;
    routeCount++;
}

// Redimensionar para o array de retorno sem posições nulas
Route[] finalRoutes = new Route[routeCount];
for (int i = 0; i < routeCount; i++) {
    finalRoutes[i] = tempRoutes[i];
}

return finalRoutes;
}
```

Análise Crítica do Desenho do Algoritmo generate

1. Por que usamos um array temporário e depois o redimensionamos?

Em Java, os arrays comuns (`Type[]`) são **estruturas estáticas e de tamanho imutável** na memória. Ao contrário de uma `List` (como `ArrayList`), não podemos fazer `.add()` dinamicamente.

Como o enunciado especifica que o retorno do método é do tipo `Route[]` e não pode conter posições vazias (nulas) nem rotas que não tenham caixas associadas, temos de calcular dinamicamente o tamanho correto. A técnica utilizada, chamada de *shrinking* (encolhimento), é a mais clássica e eficiente em Java puro para este propósito: * Alocamos espaço para o número máximo possível de rotas (`tempRoutes` com tamanho de `vehicles.length`). * Preenchemos sequencialmente e contamos os elementos válidos com `routeCount`. * Criamos o array definitivo de tamanho `routeCount` e copiamos os dados.

2. Assunção de Construtores (`RouteImpl`)

O enunciado não nos dá a assinatura do construtor da rota, apenas a interface `Route`. No entanto, para gerar rotas, precisamos de criar objetos novos. Assume-se a existência da classe concreta `RouteImpl` que implementa `Route` e aceita o veículo no construtor (`new`

`RouteImpl(currentVehicle)`). Esta assunção de classes e construtores baseados em interfaces é padrão nos exames da ESTG/P.PORTO e foi explicitada no comentário do código.

3. Ordem de Complexidade e Otimização

- O algoritmo possui uma complexidade temporal de $O(V * B * C)$, onde `v` é o número de veículos, `B` é o número de `AidBox` e `C` é o número médio de contentores em cada `AidBox`.
- Uma otimização importante foi colocada no método `hasCollectableContainer`: assim que o primeiro contentor recolhível é encontrado, o método faz um **retorno imediato** (`return true`). Esta técnica de curto-circuito previne que continuemos a iterar desnecessariamente sobre os outros contentores da caixa de ajuda, economizando ciclos de processamento de CPU.